

Datenstruktur `HashMap` in Java

Merkmale

Merkmal	Beschreibung
Schlüssel-Wert-Paar	Eine <code>HashMap</code> speichert Daten in Form von Schlüssel-Wert-Paaren, wobei jeder Schlüssel eindeutig ist.
Dynamische Größe	Die <code>HashMap</code> wächst und schrumpft dynamisch, basierend auf der Anzahl der gespeicherten Elemente.
Schneller Zugriff	Der Zugriff auf Werte erfolgt über den Schlüssel und erfolgt in durchschnittlich konstanter Zeit, $O(1)$.
Keine Ordnung	Die Reihenfolge der Elemente in einer <code>HashMap</code> ist nicht garantiert. Wenn eine geordnete Sammlung gewünscht ist, wird <code>LinkedHashMap</code> verwendet.
Null-Werte	Eine <code>HashMap</code> kann genau einen <code>null</code> -Schlüssel und mehrere <code>null</code> -Werte enthalten.
Nicht thread-sicher	<code>HashMap</code> ist nicht thread-sicher. Für Multithread-Umgebungen ist entweder Synchronisierung notwendig oder die Verwendung von <code>ConcurrentHashMap</code> .
Kein doppelter Schlüssel	Ein Schlüssel in einer <code>HashMap</code> kann nur einmal vorkommen. Beim Hinzufügen eines bestehenden Schlüssels wird der alte Wert durch den neuen ersetzt.

Methoden

Methode	Beschreibung
<code>put()</code>	Fügt ein Schlüssel-Wert-Paar zur Map hinzu. Wenn der Schlüssel bereits vorhanden ist, wird der Wert überschrieben.
<code>get()</code>	Gibt den Wert zurück, der mit dem angegebenen Schlüssel verknüpft ist.
<code>remove()</code>	Entfernt das Schlüssel-Wert-Paar, das mit dem angegebenen Schlüssel verknüpft ist.
<code>containsKey()</code>	Überprüft, ob ein bestimmter Schlüssel in der Map vorhanden ist.
<code>containsValue()</code>	Überprüft, ob ein bestimmter Wert in der Map vorhanden ist.
<code>keySet()</code>	Gibt eine <code>Set</code> -Ansicht der Schlüssel in der Map zurück.
<code>values()</code>	Gibt eine Collection-Ansicht der Werte in der Map zurück.
<code>entrySet()</code>	Gibt eine <code>Set</code> -Ansicht der Schlüssel-Wert-Paare in der Map zurück.
<code>size()</code>	Gibt die Anzahl der Schlüssel-Wert-Paare in der Map zurück.

Methode	Beschreibung
<code>isEmpty()</code>	Überprüft, ob die Map leer ist.
<code>clear()</code>	Entfernt alle Schlüssel-Wert-Paare aus der Map.
<code>putIfAbsent()</code>	Fügt das Schlüssel-Wert-Paar nur hinzu, wenn der Schlüssel noch nicht existiert.
<code>replace()</code>	Ersetzt den Wert eines bestimmten Schlüssels, wenn dieser existiert.
<code>forEach()</code>	Iteriert über alle Schlüssel-Wert-Paare in der Map und führt eine Aktion aus.

Anwendungsszenarien

Szenario	Beschreibung
Schneller Zugriff auf Daten	Wenn schnell auf Daten anhand eines eindeutigen Schlüssels zugegriffen werden muss, eignet sich die HashMap aufgrund des $O(1)$ -Zugriffs.
Zählen von Vorkommen	HashMaps eignen sich gut für das Zählen von Vorkommen von Objekten oder Wörtern, indem die Objekte als Schlüssel und die Vorkommen als Werte gespeichert werden.
Caching von Daten	HashMap ist nützlich, wenn Daten zwischengespeichert werden müssen, um schnelle Zugriffszeiten für häufig verwendete Daten zu gewährleisten.
Verwalten von Konfigurationen oder Settings	HashMaps eignen sich ideal zum Speichern von Konfigurationseinstellungen, die mit einem Schlüssel abgerufen werden können.
Verwaltung von Relationen	Wenn es notwendig ist, Beziehungen zwischen Entitäten zu modellieren, kann eine HashMap als Datenstruktur verwendet werden, bei der der Schlüssel eine Entität und der Wert eine zugehörige Entität ist.
Indexierung von Objekten	In Anwendungen, in denen Objekte anhand eines einzigartigen Identifikators (z. B. einer ID) abgerufen werden müssen, ist eine HashMap eine gute Wahl.